```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
# from datasets import load_dataset # Removed as we are loading a local CSV
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Embedding, LSTM, SpatialDropout1D, Conv1D, GlobalMaxPooling1D, Dropout, Flatten
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
from tensorflow.keras.metrics import Precision, Recall
from tensorflow.keras.utils import to_categorical
import io
from google.colab import files
import zipfile # Added for handling zip files
import os      # Added for file system operations

df = None # Initialize df to None

# Load the Movie_reviews.csv file
try:
    df = pd.read_csv('Movie_reviews.csv')
    print("Movie_reviews.csv loaded successfully.")
except FileNotFoundError:
    print("Error: 'Movie_reviews.csv' not found. Attempting to prompt for upload.")
    uploaded = files.upload() # Prompt user to upload

    # Check for directly uploaded Movie_reviews.csv
    if 'Movie_reviews.csv' in uploaded:
        df = pd.read_csv(io.BytesIO(uploaded['Movie_reviews.csv']))
        print("Movie_reviews.csv loaded successfully after direct upload.")
    else:
        found_and_loaded = False
        for filename in uploaded.keys():
            if filename.endswith('.zip'):
                print(f"Zip file '{filename}' detected. Extracting contents...")
                # Write the uploaded bytes to a file
                with open(filename, 'wb') as f:
                    f.write(uploaded[filename])
                with zipfile.ZipFile(filename, 'r') as zip_ref:
                    zip_ref.extractall('.') # Extract to current directory
                print(f"'{filename}' extracted successfully. Checking for Movie_reviews.csv or alternative CSVs...")

                # Check for CSVs within extracted files
                if os.path.exists('Movie_reviews.csv'):
                    df = pd.read_csv('Movie_reviews.csv')
                    print("Movie_reviews.csv loaded successfully after extracting zip.")
                    found_and_loaded = True
                    break
                elif os.path.exists('turkish_movie_sentiment_dataset.csv'):
                    df = pd.read_csv('turkish_movie_sentiment_dataset.csv')
                    print("'turkish_movie_sentiment_dataset.csv' loaded successfully after extracting zip.")
                    found_and_loaded = True
                    break
            elif 'turkish_movie_sentiment_dataset' in filename and filename.endswith('.csv'):
                # This handles direct upload of 'turkish_movie_sentiment_dataset.csv' or 'turkish_movie_sentiment_dataset (X
                df = pd.read_csv(io.BytesIO(uploaded[filename]))
                print(f"'{filename}' loaded successfully after direct upload.")
                found_and_loaded = True
                break

        if not found_and_loaded:
            print("Movie_reviews.csv was not uploaded directly, and no zip file or alternative CSV containing it was found.
            raise FileNotFoundError("Movie_reviews.csv not found after upload attempt.")

# Ensure df is loaded before proceeding
if df is None:
    raise FileNotFoundError("Failed to load Movie_reviews.csv from any source.")

#
# Check for the existence of 'point' column before attempting conversion
if 'point' in df.columns:
    df['label'] = df['point'].apply(lambda x: 1 if float(str(x).replace(',', '.')) > 2 else 0)  # , handle comma as decimal
elif 'label' in df.columns:
    # If 'label' column already exists, assume it's suitable or needs direct conversion
    # This part might need more sophisticated logic based on actual label values if they are not 0/1
    print("Using existing 'label' column. Assuming it contains appropriate sentiment scores or labels.")
    # If labels are string like 'positive', 'negative', further conversion needed
    # For now, assuming numeric or will be handled by subsequent steps if a problem arises
```

```
    # For now, assuming numeric or will be handled by subsequent steps if a problem arises
    # df['label'] = df['label'].apply(lambda x: 1 if x == 'positive' else 0) # Example for string labels
else:
    raise KeyError("Neither 'point' nor 'label' column found in the DataFrame. Cannot proceed with label conversion.")

# Convert 'comment' column to 'text' for consistency if it exists
if 'comment' in df.columns:
    df = df[['comment', 'label']].rename(columns={'comment': 'text'})
elif 'text' not in df.columns:
    raise KeyError("Neither 'comment' nor 'text' column found in the DataFrame for review text.")

df = df.dropna().sample(5000, random_state=42)  #

print(f"{len(df)}")
print(df['label'].value_counts())
```

```
Error: 'Movie_reviews.csv' not found. Attempting to prompt for upload.
```

Dosyaları Seç Dosya seçilmedi Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving test.csv.zip to test.csv.zip
Zip file 'test.csv.zip' detected. Extracting contents...
'test.csv.zip' extracted successfully. Checking for Movie_reviews.csv or alternative CSVs...
'turkish_movie_sentiment_dataset.csv' loaded successfully after extracting zip.
5000
label
1    4133
0     867
```

Kodlamaya başlayın veya yapay zeka ile kod oluşturun.

Kodlamaya başlayın veya yapay zeka ile kod oluşturun.

```
#
X_train, X_test, y_train, y_test = train_test_split(
    df['text'], df['label'], test_size=0.1, stratify=df['label'], random_state=42
)

# Tokenizer
vocab_size = 10000
tokenizer = Tokenizer(num_words=vocab_size, oov_token='<OOV>')
tokenizer.fit_on_texts(X_train)

train_seq = tokenizer.texts_to_sequences(X_train)
test_seq = tokenizer.texts_to_sequences(X_test)

max_length = 30
train_padded = pad_sequences(train_seq, maxlen=max_length, padding='post')
test_padded = pad_sequences(test_seq, maxlen=max_length, padding='post')

print(f"Vocab size: {len(tokenizer.word_index)}")
```

```
Vocab size: 36756
```

```
# Class Weight و Early Stopping
from sklearn.utils.class_weight import compute_class_weight
cw = compute_class_weight('balanced', classes=np.unique(y_train), y=y_train)
class_weights = {0: cw[0], 1: cw[1]}

es = EarlyStopping(monitor='val_accuracy', patience=5, restore_best_weights=True)

#ا Dense
model1 = Sequential()
model1.add(Embedding(vocab_size+1, 10, input_length=max_length))
model1.add(Flatten())
model1.add(Dense(32, activation='relu'))
model1.add(Dropout(0.25))
model1.add(Dense(1, activation='sigmoid'))

model1.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model1.fit(train_padded, y_train, epochs=50, batch_size=32, validation_split=0.1,
           callbacks=[es], class_weight=class_weights, verbose=1)

#
loss, acc = model1.evaluate(test_padded, y_test)
print(f'Dense Accuracy: {acc*100:.2f}%')
```

```
Epoch 1/50
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is depre
  warnings.warn(
127/127 ━━━━━━━━━━━━━━━━━━━━ 5s 19ms/step - accuracy: 0.5208 - loss: 0.7078 - val_accuracy: 0.5778 - val_loss: 0.6886
Epoch 2/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.7754 - loss: 0.6602 - val_accuracy: 0.8133 - val_loss: 0.4973
```

```
Epoch 3/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9000 - loss: 0.3653 - val_accuracy: 0.7444 - val_loss: 0.4708
Epoch 4/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9665 - loss: 0.1326 - val_accuracy: 0.7644 - val_loss: 0.5020
Epoch 5/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9707 - loss: 0.0857 - val_accuracy: 0.8444 - val_loss: 0.4150
Epoch 6/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9872 - loss: 0.0521 - val_accuracy: 0.8467 - val_loss: 0.4420
Epoch 7/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9902 - loss: 0.0301 - val_accuracy: 0.8378 - val_loss: 0.4788
Epoch 8/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9933 - loss: 0.0211 - val_accuracy: 0.8489 - val_loss: 0.4772
Epoch 9/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9948 - loss: 0.0176 - val_accuracy: 0.8444 - val_loss: 0.5112
Epoch 10/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9974 - loss: 0.0109 - val_accuracy: 0.8556 - val_loss: 0.5212
Epoch 11/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9901 - loss: 0.0329 - val_accuracy: 0.8467 - val_loss: 0.5448
Epoch 12/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9947 - loss: 0.0156 - val_accuracy: 0.8400 - val_loss: 0.5599
Epoch 13/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9915 - loss: 0.0248 - val_accuracy: 0.8444 - val_loss: 0.5836
Epoch 14/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.9963 - loss: 0.0118 - val_accuracy: 0.8467 - val_loss: 0.5974
Epoch 15/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.9983 - loss: 0.0060 - val_accuracy: 0.8244 - val_loss: 0.6310
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8361 - loss: 0.5524
Dense Accuracy: 82.40%
```

```
#CNN
model2 = Sequential()
model2.add(Embedding(vocab_size+1, 10, input_length=max_length))
model2.add(SpatialDropout1D(0.25))
model2.add(Conv1D(32, 5, activation='relu'))
model2.add(GlobalMaxPooling1D())
model2.add(Dense(32, activation='relu'))
model2.add(Dropout(0.25))
model2.add(Dense(1, activation='sigmoid'))

model2.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model2.fit(train_padded, y_train, epochs=50, batch_size=32, validation_split=0.1,
           callbacks=[es], class_weight=class_weights, verbose=1)

loss, acc = model2.evaluate(test_padded, y_test)
print(f'CNN Accuracy: {acc*100:.2f}%')
```

```
Epoch 1/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 4s 13ms/step - accuracy: 0.3037 - loss: 0.7064 - val_accuracy: 0.7756 - val_loss: 0.6837
Epoch 2/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.6510 - loss: 0.6911 - val_accuracy: 0.8111 - val_loss: 0.5378
Epoch 3/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8150 - loss: 0.5191 - val_accuracy: 0.7222 - val_loss: 0.5328
Epoch 4/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8827 - loss: 0.2901 - val_accuracy: 0.8089 - val_loss: 0.4347
Epoch 5/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9442 - loss: 0.1770 - val_accuracy: 0.8156 - val_loss: 0.4684
Epoch 6/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9612 - loss: 0.1181 - val_accuracy: 0.8289 - val_loss: 0.4704
Epoch 7/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9656 - loss: 0.0809 - val_accuracy: 0.8489 - val_loss: 0.4742
Epoch 8/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9777 - loss: 0.0611 - val_accuracy: 0.8333 - val_loss: 0.5476
Epoch 9/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9886 - loss: 0.0358 - val_accuracy: 0.8467 - val_loss: 0.5654
Epoch 10/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9849 - loss: 0.0373 - val_accuracy: 0.8356 - val_loss: 0.6221
Epoch 11/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - accuracy: 0.9922 - loss: 0.0268 - val_accuracy: 0.8333 - val_loss: 0.6788
Epoch 12/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9898 - loss: 0.0202 - val_accuracy: 0.8444 - val_loss: 0.6852
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8632 - loss: 0.4841
CNN Accuracy: 86.60%
```

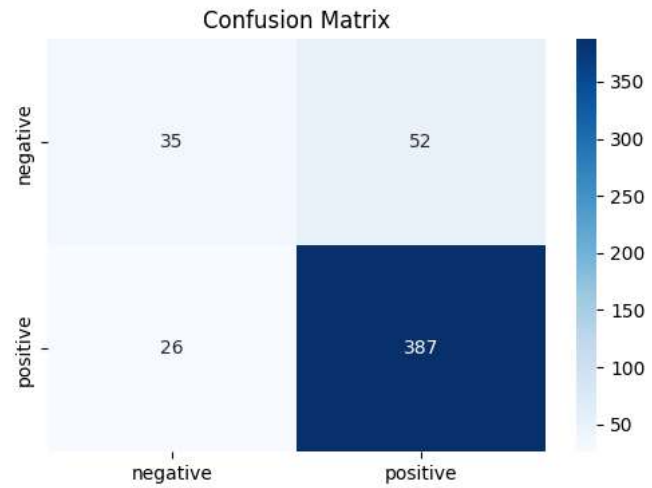
```
#lstm
model3 = Sequential()
model3.add(Embedding(vocab_size+1, 10, input_length=max_length, name='embeded'))
model3.add(SpatialDropout1D(0.25))
model3.add(LSTM(16, return_sequences=True))
model3.add(LSTM(8))
model3.add(Dropout(0.25))
model3.add(Dense(1, activation='sigmoid'))

model3.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model3.fit(train_padded, y_train, epochs=50, batch_size=32, validation_split=0.1,
           callbacks=[es], class_weight=class_weights, verbose=1)
```

```
loss, acc = model3.evaluate(test_padded, y_test)
print(f'LSTM Accuracy: {acc*100:.2f}%')
```

```
Epoch 1/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 10s 38ms/step - accuracy: 0.5877 - loss: 0.6821 - val_accuracy: 0.1644 - val_loss: 0.7937
Epoch 2/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 10s 76ms/step - accuracy: 0.5644 - loss: 0.6296 - val_accuracy: 0.6289 - val_loss: 0.6581
Epoch 3/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 10s 75ms/step - accuracy: 0.8412 - loss: 0.3841 - val_accuracy: 0.7978 - val_loss: 0.4268
Epoch 4/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 9s 70ms/step - accuracy: 0.9114 - loss: 0.2663 - val_accuracy: 0.7111 - val_loss: 0.6668
Epoch 5/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 6s 45ms/step - accuracy: 0.9376 - loss: 0.1862 - val_accuracy: 0.8356 - val_loss: 0.4911
Epoch 6/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - accuracy: 0.9508 - loss: 0.1467 - val_accuracy: 0.8400 - val_loss: 0.4727
Epoch 7/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - accuracy: 0.9699 - loss: 0.1369 - val_accuracy: 0.8222 - val_loss: 0.5674
Epoch 8/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - accuracy: 0.9744 - loss: 0.0967 - val_accuracy: 0.8422 - val_loss: 0.5561
Epoch 9/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 4s 34ms/step - accuracy: 0.9787 - loss: 0.0954 - val_accuracy: 0.8156 - val_loss: 0.6744
Epoch 10/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - accuracy: 0.9829 - loss: 0.0689 - val_accuracy: 0.8378 - val_loss: 0.6441
Epoch 11/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - accuracy: 0.9780 - loss: 0.0847 - val_accuracy: 0.8200 - val_loss: 0.7182
Epoch 12/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 6s 34ms/step - accuracy: 0.9858 - loss: 0.0693 - val_accuracy: 0.8222 - val_loss: 0.7353
Epoch 13/50
127/127 ━━━━━━━━━━━━━━━━━━━━ 3s 25ms/step - accuracy: 0.9867 - loss: 0.0556 - val_accuracy: 0.8289 - val_loss: 0.7699
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.8467 - loss: 0.5436
LSTM Accuracy: 84.40%
```

```
#
y_pred = (model3.predict(test_padded) > 0.5).astype(int).flatten()

print(classification_report(y_test, y_pred, target_names=['negative', 'positive']))

#  Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['negative', 'positive'], yticklabels=['negative', 'positive
plt.title('Confusion Matrix')
plt.show()
```

```
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 29ms/step
              precision    recall  f1-score   support

    negative       0.57      0.40      0.47        87
    positive       0.88      0.94      0.91       413

    accuracy                           0.84       500
   macro avg       0.73      0.67      0.69       500
weighted avg       0.83      0.84      0.83       500
```




```
# Embeddings
e = model3.get_layer('embeded')
weights = e.get_weights()[0]

out_v = io.open('vecs.tsv', 'w', encoding='utf-8')
out_m = io.open('meta.tsv', 'w', encoding='utf-8')

# Iterate through the tokenizer's word_index and save embeddings only for words within vocab_size
# The embedding layer 'weights' has indices from 0 to vocab_size (10000 in this case).
```

```
# tokenizer.word_index maps words to indices 1, 2, ...
# We save embeddings for OOV token (usually index 1) and then for actual words up to vocab_size.

# Check if OOV token exists and its corresponding embedding (index 1) should be saved
if tokenizer.oov_token is not None and 1 <= vocab_size: # Assuming OOV is at index 1
    oov_word_name = tokenizer.oov_token if tokenizer.oov_token != '<OOV>' else '<unk>'
    out_m.write(oov_word_name + '\n')
    out_v.write('\t'.join([str(x) for x in weights[1]]) + '\n')

# Iterate through the tokenizer's word_index items (word and its assigned index 'i')
# and save embeddings for indices within the defined vocab_size.
for word, i in tokenizer.word_index.items():
    # Ensure the word index 'i' is within the bounds of the embedding matrix (0 to vocab_size)
    # and is not the OOV token if handled separately (i > 1).
    if i > 1 and i <= vocab_size:
        vec = weights[i]
        out_m.write(word + '\n')
        out_v.write('\t'.join([str(x) for x in vec]) + '\n')

out_v.close()
out_m.close()

#
files.download('vecs.tsv')
files.download('meta.tsv')

print(" vecs.tsv و meta.tsv : https://projector.tensorflow.org/")
```

```
 vecs.tsv و meta.tsv : https://projector.tensorflow.org/
```

```
#
statement = "bu filmi kötü"  #
seq = tokenizer.texts_to_sequences([statement])
padded = pad_sequences(seq, maxlen=max_length, padding='post')

pred = model3.predict(padded)[0][0]
print(f'ornk: "{statement}"')
print(f'sınıf: {"positive" if pred > 0.5 else "negative"} (ols: {pred*100:.2f}%)')
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 75ms/step
ornk: "bu filmi kötü"
sınıf: negative (ols: 42.51%)
```

Task

Download the 'Turkish Movie Sentiment Analysis Dataset' by 'mustfkeskin' from Kaggle, load the 'Movie_reviews.csv' file into a pandas DataFrame, and ensure the DataFrame has 'comment' and 'label' columns, replacing the current data loading method.

```
Kodlamaya başlayın veya yapay zeka ile kod oluşturun.
```